

MixBytes()

P2P.org Eth2 Depositor Security Audit Report

MAY 25, 2026

Table of Contents

1. Introduction	2
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	3
1.4 Security Audit Methodology	5
Stage 1: Interim Audit	5
Stage 2: Re-audit & Mainnet Deployment Verification	6
1.5 Risk Classification	8
1.6 Summary of Findings	9
2. Findings Report	10
2.1 Critical	10
2.2 High	10
2.3 Medium	10
2.4 Low	10
L-1 Large-Deposit Withdrawal-Credentials Sanity Check Is Unnecessarily Weak	10
L-2 Duplicate Pubkeys In A Batch Can Top Up The Same Validator	12
L-3 Hardcoded DepositContract Address Makes Deployment Chain-Sensitive	14
L-4 Paused And Renounced Ownership State Permanently Disables The Wrapper	15
L-5 Amount Lower Bound And Gwei Alignment Are Only Checked Downstream	16
L-6 Pause And Unpause Can Be External	17
3. About MixBytes	18

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

`P2pEth2Depositor` is a standalone Solidity batch wrapper around the canonical Ethereum `DepositContract`. It accepts a batch of validator pubkeys, signatures, deposit data roots, one shared withdrawal credentials value, and a uniform per-validator ETH amount, then forwards one deposit per validator to the `DepositContract` address set at deployment. The contract is non-custodial in normal operation: direct ETH transfers are rejected, and successful deposits forward all received ETH to the `DepositContract`. Owner privileges are limited to pausing and unpausing the permissionless `deposit()` entry point.

The review focused on the wrapper's core deposit logic and its direct interaction with the canonical Ethereum `DepositContract`. The team also worked through the standard security checklist, including access control, external calls, ETH accounting, input validation, denial-of-service conditions, and operational monitoring assumptions. The review included the following project-specific attack vectors:

- **Beacon Deposit Data Integrity.** Reviewed whether the wrapper can prevent malformed or inconsistent validator deposit data, including mismatched pubkeys, signatures, withdrawal credentials, amounts, and deposit data roots, from being forwarded to the canonical `DepositContract`.
- **EIP-7251 Large-Validator Semantics.** Checked whether deposits above 32 ETH are constrained to the expected compounding withdrawal-credential model and whether the wrapper's 2048 ether cap matches consensus-layer effective-balance behavior.
- **Front-Running And First-Deposit Ordering Assumptions.** Considered whether validator pubkeys can be pre-registered with different withdrawal credentials before intended deposits, and whether this risk is mitigated by trusted validator operators or requires integration-level controls.

The canonical Ethereum `DepositContract` was out of scope of this audit; it was only reviewed and its documentation was studied.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Minimal pass-through design.** The contract has a narrow surface area and does not maintain deposit accounting, escrow balances, or user mappings. This keeps the core logic easy to reason about, but also means that all semantic correctness of deposit data must be guaranteed by off-chain tooling and by the canonical `DepositContract`.
- **Deposit data correctness is caller-owned.** The wrapper only partially validates caller-supplied deposit data. Incorrect deposit data can make deposited funds permanently inaccessible at the consensus layer, so callers must validate it off-chain before submission.

- **Additional sanity checks may reduce operator mistakes.** The wrapper includes only narrow boundary checks. The project should consider whether to add lightweight validation for obviously unsafe withdrawal credentials, such as credentials that resolve to a dead-end destination like the zero address or the depositor contract itself.
- **Operational assumptions are important.** Safe deployment depends on using the correct `DepositContract` address for the target network. Operational workflows should also account for owner governance and pause/unpause handling.
- **Validator deposit front-running remains a threat-model consideration.** In Ethereum staking, withdrawal credentials are fixed by the first valid Beacon deposit for a validator pubkey, so an entity controlling the validator signing key could theoretically pre-register a pubkey with different withdrawal credentials before a later top-up. For the current project, the client confirmed that validators created through this depositor are expected to be operated by trusted/P2P-controlled nodes only; if that trust assumption changes, additional pre-deposit verification should be introduced.
- **Monitoring must use canonical logs.** The wrapper's `DepositEvent` is aggregate-only and must not be treated as the source of truth for validator deposits. Indexers should reconcile it against the canonical `DepositEvent` logs emitted by the Ethereum `DepositContract`.

1.3 Project Overview

Summary

Title	Description
Client	P2P.org
Category	Staking
Project	Eth2 Depositor
Type	Solidity
Platform	EVM
Timeline	19.05.2026 – 21.05.2026

Scope of Audit

File	Link
src/P2pEth2Depositor.sol	src/P2pEth2Depositor.sol

Versions Log

Date	Commit Hash	Note
19.05.2026	192239e3512d6723bb7cdf96cf2aadfa664c4d27	Initial Commit
21.05.2026	a165208e8bc2b619d2e787d7a4b71223748f3e7f	Re-audit Commit

Mainnet Deployments

File	Address	Blockchain
P2pEth2Depositor	0xa6cf239220a8a9939d3207dfd8f26cd5e4f36ae6	Ethereum

1.4 Security Audit Methodology

Stage 1: Interim Audit

Project Architecture Review

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM, HOW THEY INTERACT ARCHITECTURALLY, AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

Activities include:

- understanding overall system design and contract interactions
- mapping responsibilities of each component and protocol workflows
- conducting a thorough line-by-line review of contracts and modules
- tracing execution paths and mapping state transitions
- identifying trust boundaries and external integrations
- forming an independent architectural understanding directly from code before validating documentation
- running the project test suite to observe implementation behavior and encoded invariants
- reviewing documentation, specifications, READMEs, and deployment guides and reconciling them with code

Adversarial Code Review

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

Approach includes:

- analyzing the codebase from an attacker's perspective, focusing on what can fail rather than only confirming intended behavior
- searching for overlooked edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- attempting to violate protocol invariants and identifying scenarios where system guarantees break
- analyzing economic attack surfaces including liquidation logic, oracle dependencies, vault accounting, and cross-contract interactions
- writing targeted tests, modelling adversarial scenarios, applying mutation testing and fuzzing, and developing proof-of-concept exploits
- conducting independent exploit path discovery by each auditor to reduce anchoring bias
- reviewing high-risk code collaboratively in pair-auditing sessions led by the Team Lead

Systematic Vulnerability Analysis

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

Activities include:

- reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- analyzing common DeFi attack classes such as reentrancy, accounting manipulation, oracle manipulation, privilege escalation, and state desynchronization
- using proprietary AI tooling to surface candidate issues across broader attack vectors
- manually validating and triaging AI-generated candidates

Consolidation of Auditors' Reports

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

Activities include:

- cross-checking findings across auditors
- consolidating and deduplicating issues
- resolving discrepancies in severity assessments
- using proprietary AI tooling to assist with report drafting and consistency checks
- manually reviewing and finalizing the report narrative
- preparing the interim audit report for client review

Stage 2: Re-audit & Mainnet Deployment Verification

Re-audit

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THAT THE UPDATED CODE DOES NOT INTRODUCE NEW VULNERABILITIES.

Activities include:

- confirming each remediation matches the original recommendation
- documenting rationale for any unresolved findings
- verifying modified logic and integration points remain secure
- executing the project's tests after remediation, including targeted coverage of fixed areas
- using proprietary AI tooling on the updated codebase to surface potential new issues

Mainnet Deployment Verification

OBJECTIVE: PERFORM FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC AUDIT REPORT.

Activities include:

- verifying deployed contract bytecode against audited source code across networks
- confirming deployed bytecode matches the build produced from the audited commit using identical compiler settings
- reviewing constructor and initializer arguments used in deployment
- verifying proxy deployment order, initialization flow, and admin configuration
- ensuring implementations are not left uninitialized or exposed to initialization front-running
- publishing the final report after alignment between client and audit team

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.
- **Medium** – Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.
- **Low** – One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

Likelihood

- **High** – An event with an estimated 50-60% probability of occurring within a year, which can be triggered by any actor (e.g., due to a market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the contract owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed in order to minimize potential risks.
- **Medium** – Recommended to be fixed to improve security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and efficiency.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and the issue no longer impacts the security of the protocol.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but the issue has not been fully resolved.
- **Acknowledged** – The recommended fixes have not been implemented, and the finding remains unresolved or has been accepted by the project team without code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	0
Low	6

Findings Statuses

ID	Finding	Severity	Status
L-1	Large-Deposit Withdrawal-Credentials Sanity Check Is Unnecessarily Weak	Low	Fixed
L-2	Duplicate Pubkeys In A Batch Can Top Up The Same Validator	Low	Acknowledged
L-3	Hardcoded DepositContract Address Makes Deployment Chain-Sensitive	Low	Fixed
L-4	Paused And Renounced Ownership State Permanently Disables The Wrapper	Low	Fixed
L-5	Amount Lower Bound And Gwei Alignment Are Only Checked Downstream	Low	Fixed
L-6	Pause And Unpause Can Be External	Low	Fixed

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

Not Found

2.4 Low

L-1	Large-Deposit Withdrawal-Credentials Sanity Check Is Unnecessarily Weak		
Severity	Low	Status	Fixed in 0f37cf5b

Description

The contract is a thin batch wrapper over the canonical Ethereum DepositContract and is not intended to perform full validation of all deposit data. The caller remains responsible for generating internally consistent pubkeys, signatures, withdrawal credentials, amounts, and deposit data roots.

Nevertheless, the wrapper implements a sanity check for large deposits. It allows per-validator deposits up to 2048 ether, but for deposits above 32 ether it only rejects withdrawal credentials with prefix 0x01:

```
if (amount > collateral) {
    require(withdrawal_credentials[0] != ETH1_WITHDRAWAL_PREFIX,
            "P2pEth2Depositor: large deposit cannot use 0x01");
}
```

EIP-7251 introduces 0x02 compounding withdrawal credentials and raises the max effective balance to 2048 ETH only for compounding validators. Therefore, if the wrapper provides a sanity check for large deposits, the check should reject every prefix except 0x02. The current negative check is unnecessarily weak: it blocks 0x01, but still allows 0x00 and unknown prefixes even though they are not the intended compounding credentials for deposits above 32 ETH.

For example, an operator can submit a batch with amount = 100 ether and withdrawal_credentials[0] == 0x00. The wrapper accepts the batch because 0x00 != 0x01, and the DepositContract accepts the opaque credentials if the roots are consistent. The resulting issue affects the caller that supplied the unintended deposit data: the validator may be created

without the intended compounding semantics, so capital above 32 ETH may not provide the expected effective-balance benefit.

The impact is limited because the contract is permissionless and non-custodial: an incorrect caller-supplied credential harms the caller's own deposit workflow rather than other users or funds held by the wrapper. For this reason, the issue is classified as Low severity.

Recommendation

Replace the negative check with a positive sanity check that accepts only the compounding prefix for deposits above 32 ETH:

```
if (amount > collateral) {
    require(withdrawal_credentials[0] == 0x02,
        "P2pEth2Depositor: large deposit requires 0x02");
}
```

L-2	Duplicate Pubkeys In A Batch Can Top Up The Same Validator		
Severity	Low	Status	Acknowledged

Description

The wrapper validates array lengths and the byte length of each pubkey, but it does not check that `pubkeys` are unique within a batch:

```
for (uint256 i = 0; i < validatorCount; ++i) {
    require(pubkeys[i].length == pubkeyLength,
        "P2pEth2Depositor: wrong pubkey");
    require(signatures[i].length == signatureLength,
        "P2pEth2Depositor: wrong signatures");

    depositContract.deposit{value: amount}({
        pubkeys[i],
        withdrawal_credentials,
        signatures[i],
        deposit_data_roots[i]
    });
}
```

The canonical `DepositContract` also does not reject a repeated validator pubkey. At the consensus layer, the first deposit for a new pubkey can register the validator, while subsequent deposits for the same pubkey are treated as additional balance for that existing validator rather than as new validators.

As a result, a malformed batch can contain the same pubkey more than once and still execute successfully if each deposit data root is internally consistent. The wrapper will emit `validatorCount` equal to the number of submitted entries, but fewer distinct validators may be created. This can cause operational accounting errors and unintended top-ups, especially when off-chain systems assume that every batch entry corresponds to a distinct validator.

The same behavior also weakens the wrapper's `amount <= maxCollateral` sanity check. The check limits each submitted deposit entry, not the cumulative amount addressed to a distinct validator pubkey. A batch can therefore include the same pubkey multiple times, with each entry individually below `2048 ether`, while the total amount credited to that validator exceeds the intended per-validator cap. Any balance above the consensus-layer maximum effective balance does not increase validator weight and may only become useful or withdrawable according to the consensus-layer withdrawal rules.

The impact is limited because the issue affects caller-supplied deposit data and does not let an external actor steal funds from the wrapper. However, a depositor can unintentionally allocate ETH to an already included validator instead of creating the expected number of validators, so the issue is classified as Low severity.

Recommendation

Reject duplicate pubkeys in `deposit()`

Client's Commentary:

The project team acknowledges the issue and considers duplicate-pubkey validation to be an operator/integration-level sanity check

rather than an on-chain security requirement. Enforcing uniqueness on-chain would either add significant gas overhead for large batches or require changing the off-chain infrastructure, so the team decided to keep this validation off-chain.

L-3	Hardcoded DepositContract Address Makes Deployment Chain-Sensitive		
Severity	Low	Status	Fixed in 0f37cf5b

Description

The contract hardcodes the Ethereum mainnet DepositContract address:

```
IDepositContract public constant depositContract =
    IDepositContract(0x00000000219ab540356cBB839Cbe05303d7705Fa);
```

This address is correct for Ethereum mainnet and for networks that deliberately use the same deposit contract address. However, the constructor does not assert `block.chainid`, and the contract does not verify that the target address contains the expected DepositContract code. If the wrapper is deployed to a network with a different canonical deposit contract address, `deposit()` calls can fail or interact with a contract that does not provide the expected beacon-deposit semantics.

Recommendation

Add deployment checks for `block.chainid`. For multi-network deployments, use an explicit constructor or immutable deposit contract address per network.

L-4	Paused And Renounced Ownership State Permanently Disables The Wrapper		
Severity	Low	Status	Fixed in 0f37cf5b

Description

The contract inherits `renounceOwnership()` from OpenZeppelin `Ownable` and does not override it. If the owner calls `pause()` and then calls `renounceOwnership()`, `_paused == true` and `owner() == address(0)`. Since `unpause()` is protected by `onlyOwner`, no caller can ever unpause the contract. No ETH is locked in the wrapper under normal operation, but `deposit()` will revert forever with the pause guard. The only practical remediation is deploying a new wrapper and migrating operational workflows. This can also affect downstream integrations that store the wrapper address as an `immutable` value or otherwise cannot quickly update the configured depositor address: the wrapper remains deployed but non-functional, so those integrations may require redeployment or an operational migration.

Recommendation

Consider overriding `renounceOwnership()` to revert when `paused() == true`, or disable ownership renouncement entirely if ownerless operation is not required.

L-5	Amount Lower Bound And Gwei Alignment Are Only Checked Downstream		
Severity	Low	Status	Fixed in 0f37cf5b

Description

The wrapper validates `amount <= maxCollateral` and `msg.value == amount * validatorCount`, but it does not validate the lower bound or gwei alignment required by the canonical DepositContract:

```
require(amount <= maxCollateral, "P2pEth2Depositor: amount is above maximum");
uint256 totalAmount = amount * validatorCount;
require(msg.value == totalAmount, "P2pEth2Depositor: ETH sent must equal sum of amounts");
```

The canonical DepositContract requires `msg.value >= 1 ether` and `msg.value % 1 gwei == 0`. Values such as `amount = 0`, `amount = 1 wei`, or non-gwei-aligned amounts can pass the wrapper's pre-loop checks and revert only on the first external DepositContract call. The transaction reverts atomically and ETH is returned, but gas is wasted and the revert reason comes from the downstream contract.

Recommendation

Mirror the canonical DepositContract amount checks in the wrapper before the external-call loop:

```
require(amount >= 1 ether, "P2pEth2Depositor: amount below minimum");
require(amount % 1 gwei == 0, "P2pEth2Depositor: amount not gwei-aligned");
```

L-6	Pause And Unpause Can Be External		
Severity	Low	Status	Fixed in 0f37cf5b

Description

The `pause()` and `unpause()` functions are declared as `public`, but they are only used as externally callable owner actions:

```
function pause() public onlyOwner {
    _pause();
}

function unpause() public onlyOwner {
    _unpause();
}
```

No internal function in the wrapper calls either function. Keeping them `public` is functionally correct, so this is a code-style observation. Declaring them `external` would better reflect the intended owner-only API.

Recommendation

Consider changing both functions to `external`:

```
function pause() external onlyOwner {
    _pause();
}

function unpause() external onlyOwner {
    _unpause();
}
```

3. About MixBytes

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems.

The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle – from early architecture design to production audits and ongoing protocol evolution.

Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including **Lido**, **Aave**, **Curve**, **1inch**, **OKX**, **Mantle**, **Fluid**, **Gearbox**, **Resolv**, and others, helping teams identify vulnerabilities, strengthen protocol design, and improve the robustness of decentralized systems.

To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review. These tools augment the work of senior auditors but do not replace expert analysis.

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

- **Design Review.** Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.
- **AI Tooling.** AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.
- **Smart Contract Audit.** Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.
- **Security Retainer.** Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

Contacts



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



hello@mixbytes.io